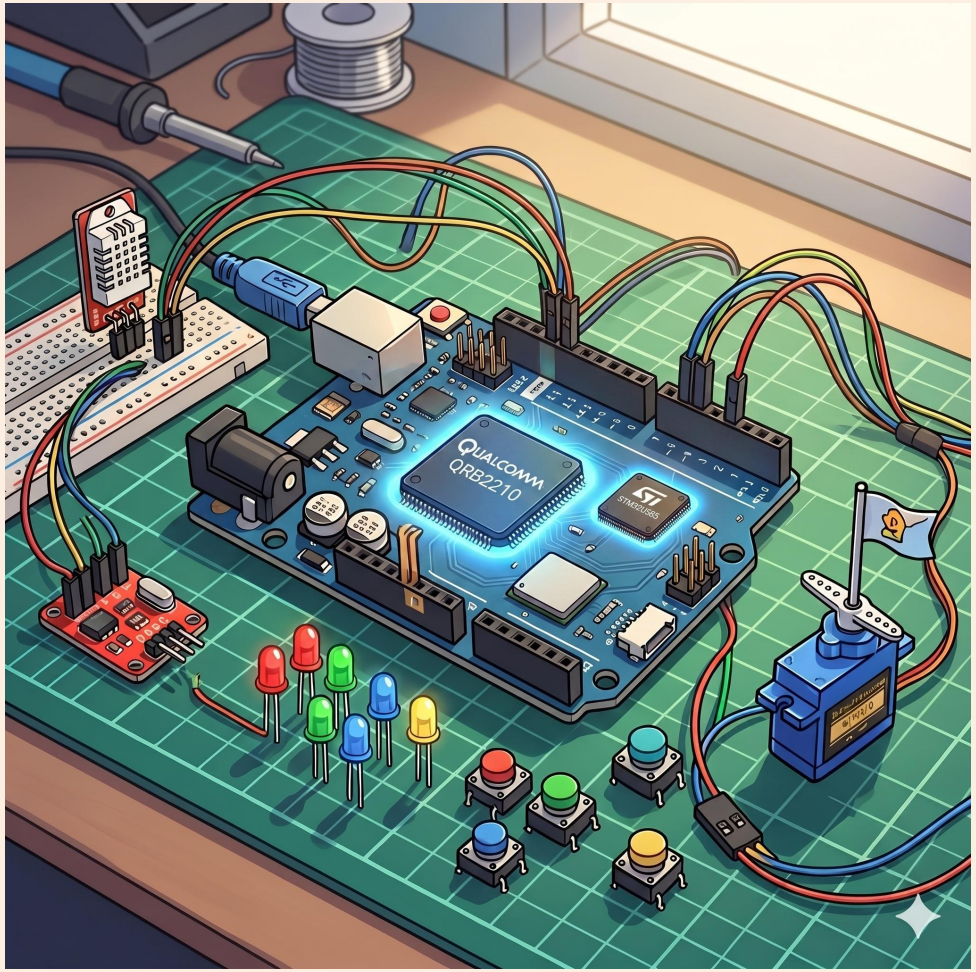
CHAPTER 4 · IESTI05 EDGE AI ENGINEERING

GenAI Meets the Real World

A dual-brain dengue risk classifier

> systemd · Bridge RPC · Flask · sensors in, actuators out

Marcelo Rovai · UNIFEI

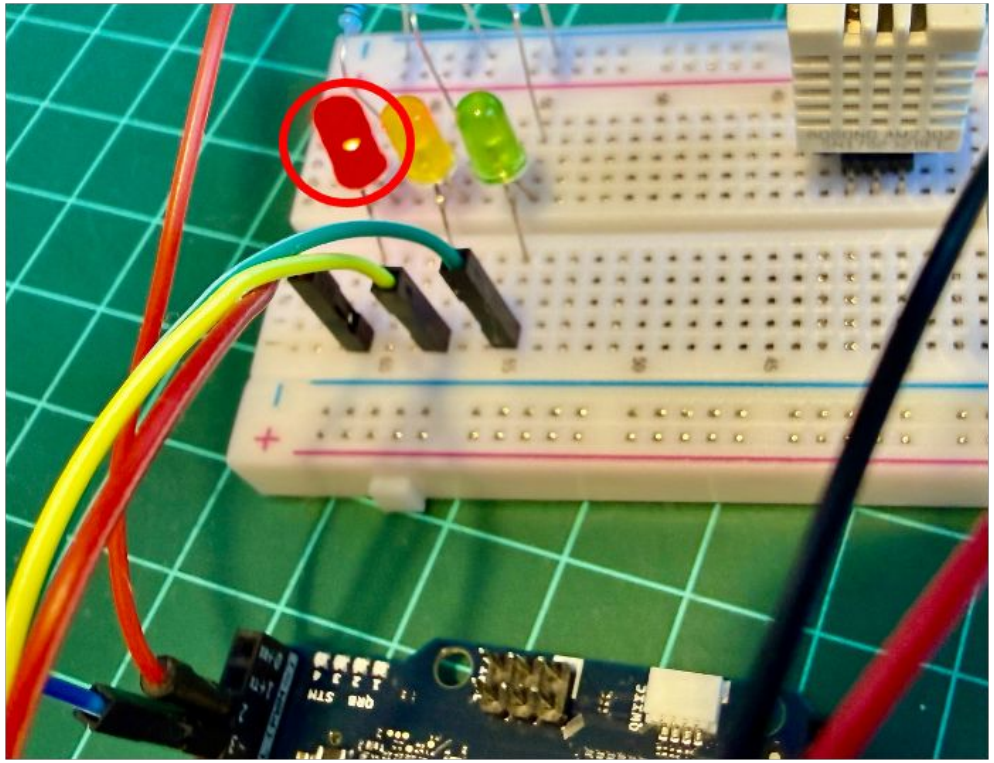


AIEng4D

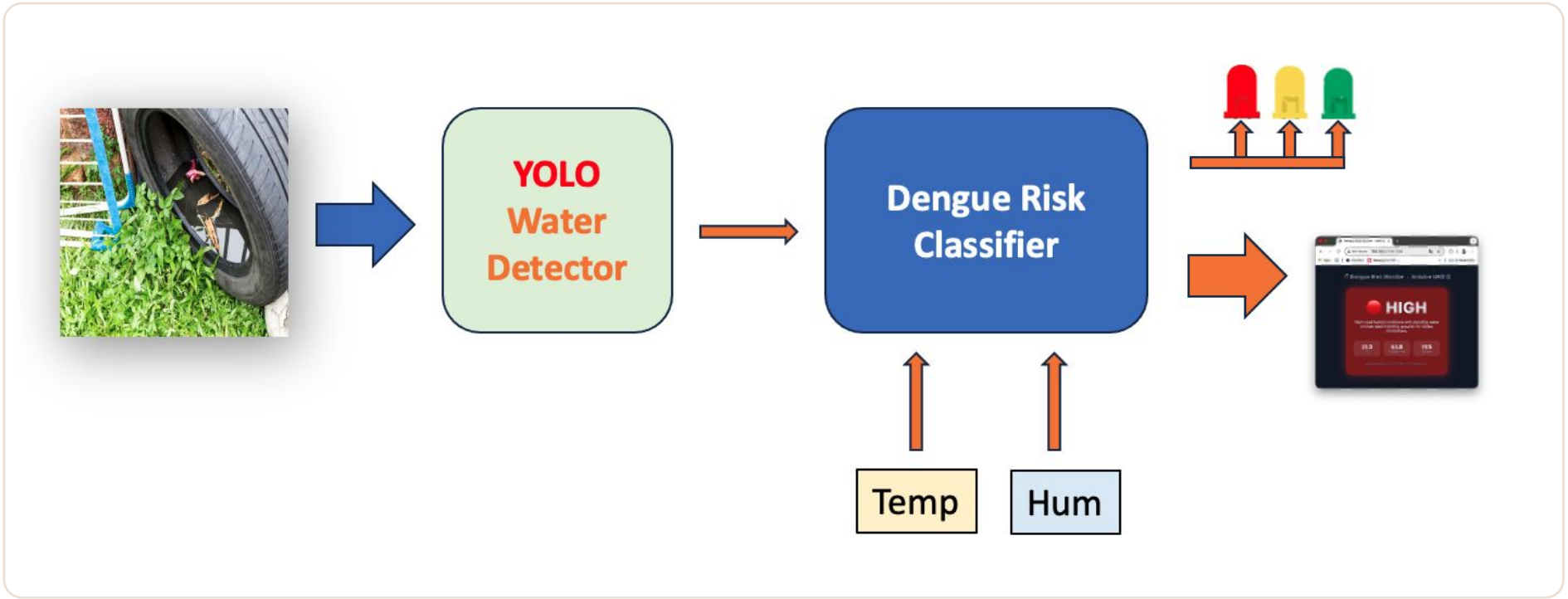
THE PROJECT

A language model that switches an LED.

Temperature and humidity from a DHT22, plus a water-presence signal. The model returns low, medium or high risk with a one-sentence reason — and the MCU turns on the matching colour.



The button is standing in for a camera



A YOLO detector spots stagnant water; that becomes one input to the classifier

Temperature and humidity are the other conditions Aedes larvae need. Chapter 3's vision pathway can replace the button without changing anything else.

THE ROUTE

Six stages, one working appliance at the end

1. **The service** — systemd: starts at boot, restarts on failure, survives a reboot.
2. **Architecture** — Three data paths, one classify(). The whiteboard slide.
3. **Python side** — One inference function, two interfaces — and one networking trick.
4. **MCU side** — A DHT22, a button, three LEDs, one Bridge call every thirty seconds.
5. **Running it** — Three LEDs, a serial monitor, and a verdict you can see across the room.
6. **Limits** — End-to-end latency, why the 2B wins here, and what still breaks.

WHAT CHANGES HERE

Three things Chapters 2 and 3 did not do

1

llama-server becomes a service

A systemd unit that starts at boot and restarts on failure. Not a foreground process someone must remember to launch.

2

The loop closes through hardware

The MCU reads real sensors and drives real LEDs. You finally see WHY you want both processors.

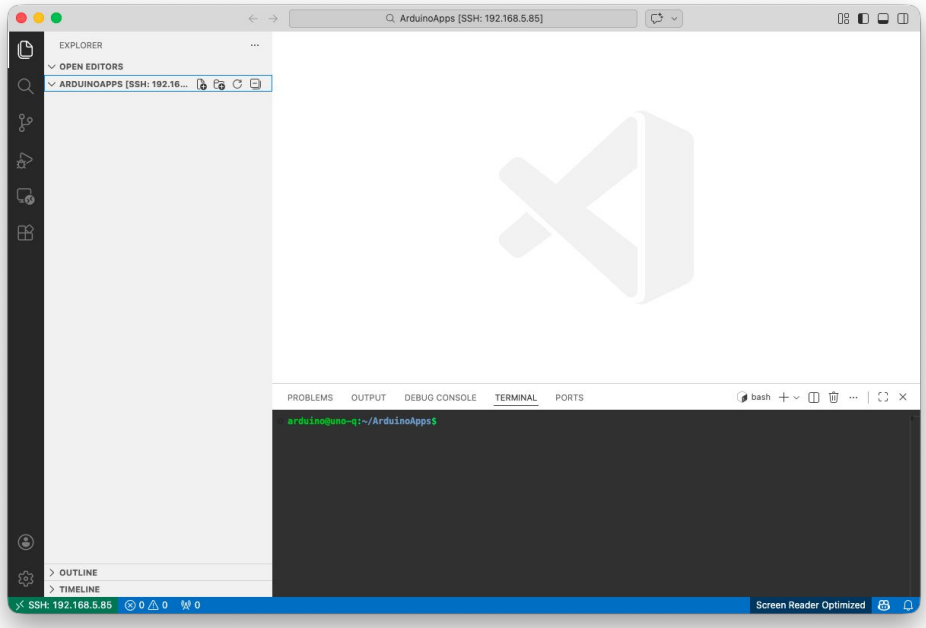
3

The SLM becomes a microservice

A Flask endpoint exposes the same classify() over HTTP to phones, browsers and other boards. One logic path, two interfaces.

Prerequisites: Ch.1 for SSH, Ch.2 for a built llama.cpp and a GGUF. Ch.3 is useful but not required.

Editing three files across two languages



VS Code Remote-SSH, editing directly on the board

Not required — nano and scp work fine. But with app.yaml, main.py and sketch.ino in two languages, Remote-SSH is a meaningfully better workflow.

STAGE 1 OF 6

llama-server as a service

1

A systemd unit that starts at boot, restarts on failure, and survives a reboot.

SYSTEMD

The service unit

```
[Unit]
Description=llama.cpp server (Qwen3.5-0.8B)
After=network-online.target

[Service]
User=arduino
WorkingDirectory=/home/arduino/llama.cpp
ExecStart=/home/arduino/llama.cpp/build/bin/llama-server \
  -m /home/arduino/models/Qwen3.5-0.8B-Q8_0.gguf \
  --host 0.0.0.0 --port 8081 -c 1024 -t 4 \
  --reasoning off --reasoning-budget 0 \
  --alias qwen3.5-0.8b
Restart=on-failure
RestartSec=10
Nice=-5

[Install]
WantedBy=multi-user.target
```

Moved the binaries to:
~/llama-runtime in Ch.2?

Adjust the paths and add
Environment=LD_LIBRARY_PATH=
/home/arduino/llama-runtime.

SYSTEMD

The four lines that only matter in a service

1. **Restart=on-failure**
   If the process is OOM-killed, systemd brings it back instead of leaving the endpoint dead.
2. **RestartSec=10**
   Ten seconds before retrying, so a crash loop does not hammer the board.
3. **Nice=-5**
   Slightly higher scheduling priority than user-space apps. Do not go below -5 or you risk starving the kernel.
4. **--host 0.0.0.0**
   Deliberate: a loopback-bound server is invisible from inside the App Lab container. Section 3 explains why that matters.

> -c 1024 is enough for structured classification. Larger values eat KV-cache RAM you do not have.

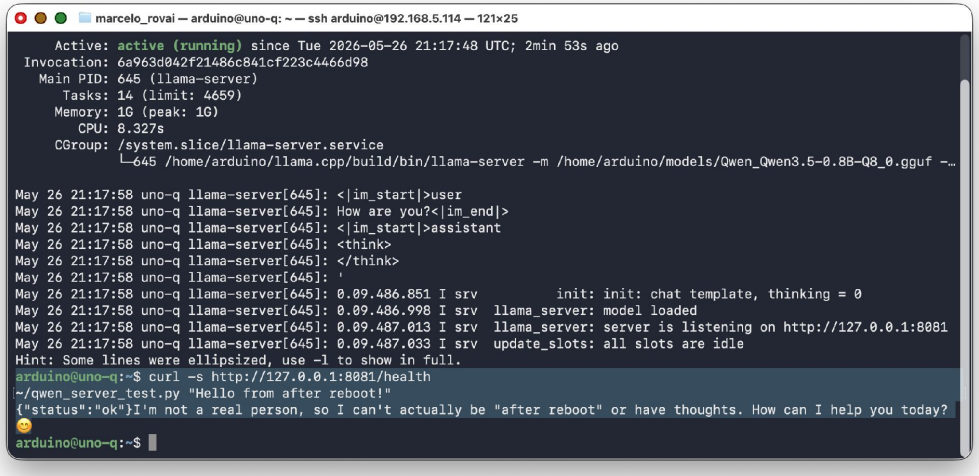
VERIFICATION

Enable it, then prove it survives a reboot

```
$ sudo systemctl daemon-reload
$ sudo systemctl enable --now llama-server.service
$ systemctl status llama-server --no-pager
$ journalctl -u llama-server -f

# wait for 'HTTP server listening', then Ctrl+C
$ sudo reboot

# after it comes back:
$ curl -s http://127.0.0.1:8081/health
  {"status":"ok"}
```



Alive again after a power cycle

> Do the reboot in class. A service that works until the power cycles is not a service.

STAGE 2 OF 6

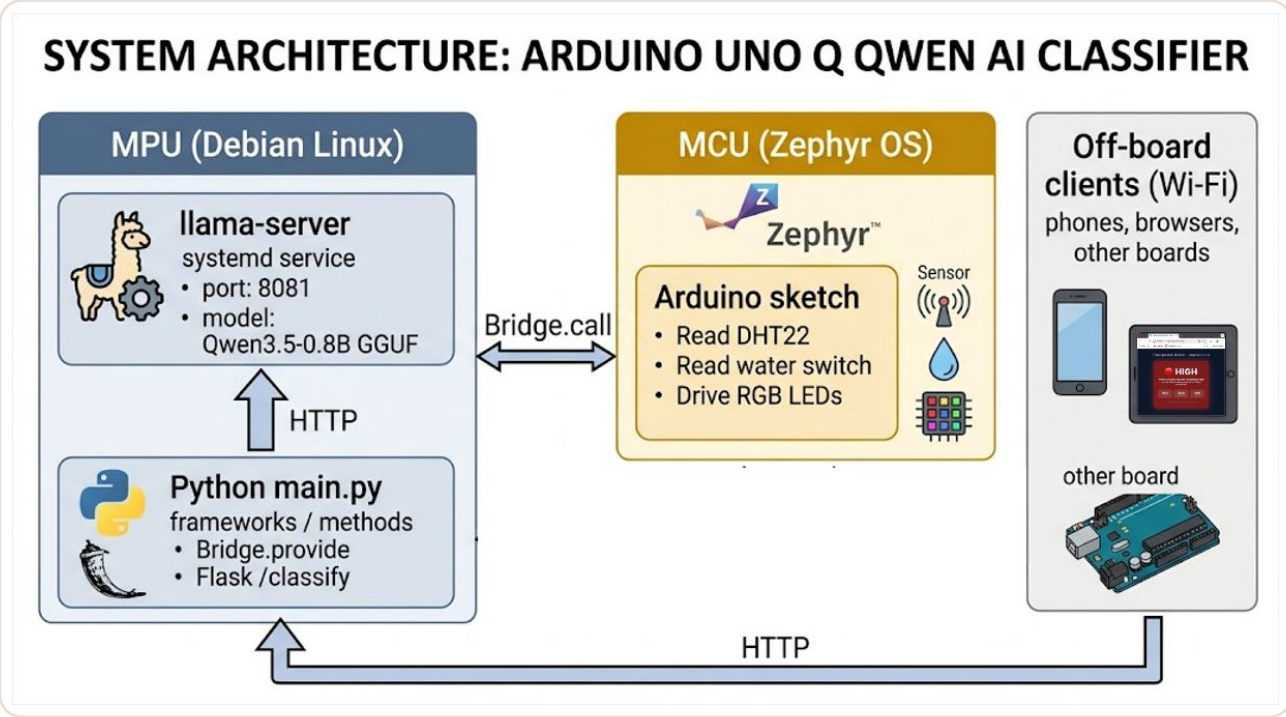
The dual-brain architecture

2

Three data paths, one classify() function.

ARCHITECTURE

Three data paths, one logic path



MCU ⇄ Python ⇄ llama-server, with Flask hanging off the side

Bridge.call("classify", 29.5, 82, true) → POST /v1/chat/completions → {"risk":"high"} → 2.0 → LED red

THE DESIGN

The same classify() powers all three paths.

That is the point — separation between interface and logic, not three parallel implementations of the same thing.

STAGE 3 OF 6

The Python side

3

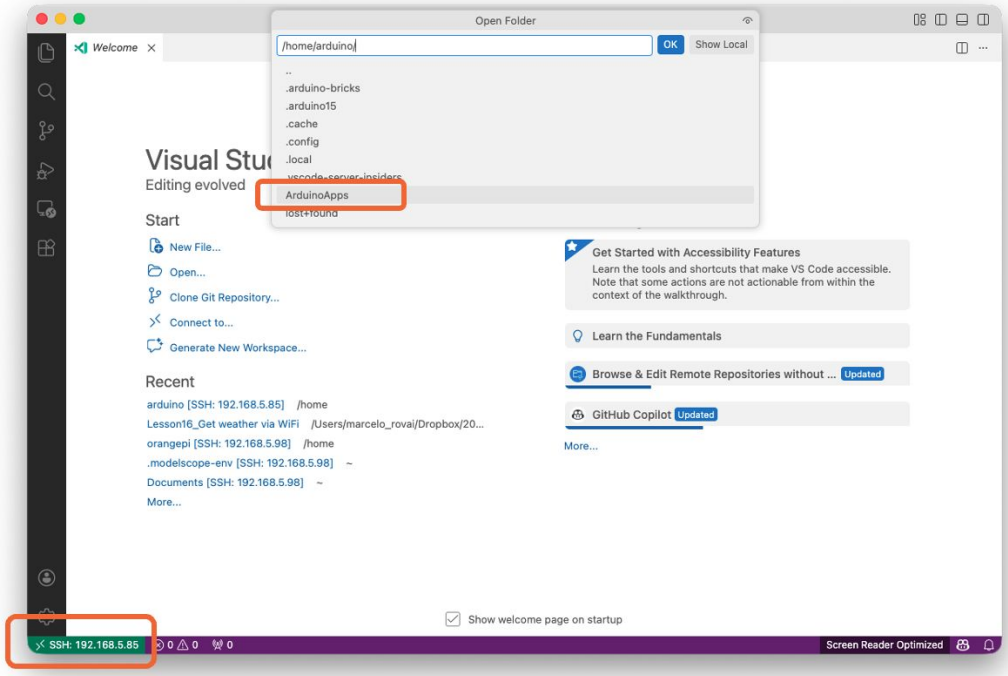
One inference function, two interfaces, and one networking trick that trips everybody up.

SCAFFOLDING

The app skeleton

```
$ cd ~/ArduinoApps
$ arduino-app-cli app new "risk-classifier"
$ cd risk-classifier

risk-classifier/
├── app.yaml
├── python/
│   ├── main.py
│   └── requirements.txt
└── sketch/
    ├── sketch.ino
    └── sketch.yaml
```



A new app, scaffolded

requirements.txt: flask==3.0.3 and requests==2.32.3 — nothing else is new in this chapter.

SCAFFOLDING

The one line in app.yaml that is load-bearing

```
name: Risk Classifier (SLM + Bridge)
description: "Dengue risk classification via Bridge RPC"
icon: 🦟
version: "1.0.0"
ports:
  - 7000
bricks: []
```

> Without ports: [7000], Flask binds inside the container only and is invisible from outside the board — and the symptom looks exactly like a firewall problem.

THE NETWORKING TRICK

Inside the container, 127.0.0.1 is not the board.

Your app runs in a container. The model runs on the host. Loopback inside the container is not loopback on the board — so you have to find the host's address at runtime.

THE NETWORKING TRICK

Find the host from inside the container

```
def _host_gateway():
    """The default gateway in /proc/net/route is the host
       as seen from this container."""
    with open("/proc/net/route") as f:
        for line in f.readlines()[1:]:
            fields = line.strip().split()
            if fields[1] == "00000000" and \
                    int(fields[3], 16) & 2:
                return socket.inet_ntoa(
                    struct.pack("<L", int(fields[2], 16)))
    return "127.0.0.1"   # fallback outside a container

LLM_HOST = _host_gateway()
LLM_URL  = f"http://{LLM_HOST}:8081/v1/chat/completions"
```

Two ports, two things:

8081 is the model,
7000 is your app.

Confusing them is the most common wrong turn in this chapter and the next.

PROMPTING

A system prompt and two few-shots

```
SYSTEM_PROMPT = (
  "You are an environmental risk classifier for dengue
   surveillance. Given temperature (Celsius), humidity
   (%), and whether standing water is reported, output
   ONLY a JSON object with two fields: risk (one of:
   low, medium, high) and reason (one short sentence,
   max 20 words). No prose outside the JSON.")

FEW_SHOTS = [
  {"role": "user",      "content": '{"temp_c": 18.0,
     "humidity_pct": 40, "standing_water": false}'},
  {"role": "assistant", "content": '{"risk":"low",
     "reason":"Cool and dry, no water."}'},  ... ]
```

Few-shots live in code, not in the chat template. That keeps the template clean and lets you swap models without rewriting prompts.

PROMPTING

response_format json_object is not a hint.

It is a real grammar constraint on token selection — the decoder cannot emit anything that is not valid JSON. Combined with two few-shots, it is what makes a 0.8B model reliable enough to drive an actuator.

THE DESIGN

One inference path, two interfaces

```
def classify(temp_c, humidity_pct, standing_water):
    """Inner classifier. Returns the full verdict dict."""
    content, ms = call_llm(build_messages(payload))
    try:
        verdict = parse_verdict(content)
    except (json.JSONDecodeError, ValueError):
        content, ms = call_llm(messages, max_tokens=60)
        verdict = parse_verdict(content)    # one retry
    return verdict

def classify_risk(temp_c, humidity_pct, standing_water):
    """Bridge-facing: 0=low, 1=medium, 2=high."""
    v = classify(temp_c, humidity_pct, standing_water)
    return RISK_CODE.get(v.get("risk"), -1.0)

Bridge.provide("classify", classify_risk)
App.run(user_loop=loop)     # the loop itself is idle
```

rpc.result() only handles primitives, so the Bridge wrapper collapses the verdict to a number. HTTP clients get the whole dict.

STAGE 4 OF 6

The MCU side

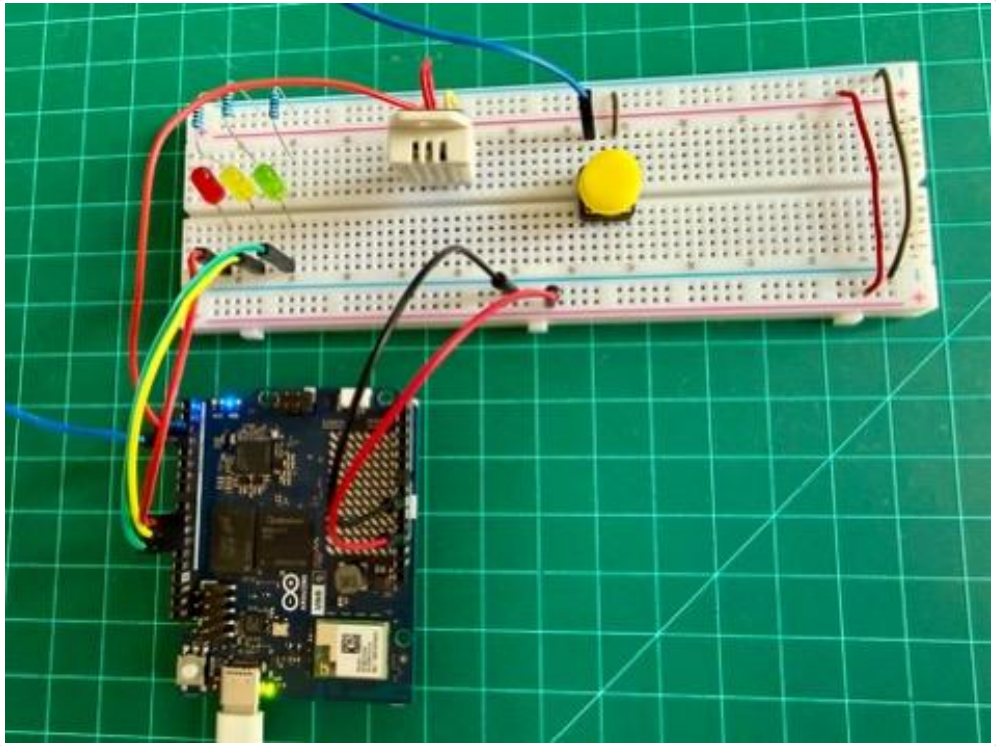
4

A DHT22, a button, three LEDs — and one Bridge call every thirty seconds.

BEFORE YOU TOUCH A JUMPER

3.3 V. Not 5 V.

The STM32U585's GPIO is 3.3 V tolerant only. A DHT22 on the 5 V rail will damage the pin it is wired to — and the failure is silent until that pin stops working.



HARDWARE

The wiring

```
Red LED     : D9  → LED → 220Ω → GND    (high risk)
Yellow LED  : D10 → LED → 220Ω → GND    (medium)
Green LED   : D11 → LED → 220Ω → GND    (low risk)

DHT22:
  VCC  → 3.3V       (NOT 5V)
  GND  → GND
  DATA → D2
  10kΩ pull-up between DATA and VCC

Button: one leg → D3, other leg → GND
```

Install the DHT library

> The button stands in for a water-presence sensor — or, later, a camera.

SKETCH.INO

One Bridge call, every thirty seconds

```
void loop() {
  updateButton();
  if (millis() - lastReading < READING_PERIOD_MS) return;
  lastReading = millis();

  float temp_c       = dht.readTemperature();
  float humidity_pct = dht.readHumidity();

  if (isnan(temp_c) || isnan(humidity_pct)) {
    setLEDs(true, true, false);   // R+Y = sensor error
    return; }

  setLEDs(true, true, true);      // all three = thinking

  float risk_f = -1.0f;
  RpcCall rpc = Bridge.call("classify", temp_c,
                            humidity_pct, water_state);
  if (rpc.result(risk_f)) { /* drive the matching LED */ }
}
```

Bridge.call() returns an RpcCall object. Do not assign it to a String and do not use >>. Use .result(var), which returns true on success.

SKETCH.INO

The LED states, and why each one exists

1

Boot: blink three times, then green

A visible 'I am alive' before any inference happens. Green is the ready state, not a verdict.

2

All three on = thinking

Inference takes 6–12 seconds. Without a busy state the board looks frozen and students press the button again.

3

Red + yellow = the DHT22 read failed

A distinct pattern for sensor failure, so a wiring problem never gets mistaken for a model problem.

4

One LED after each cycle

Green low, yellow medium, red high — legible from across the room, which is the point of doing this in hardware.

STAGE 5 OF 6

Running it

5

Three LEDs, a serial monitor, and a log line every thirty seconds.

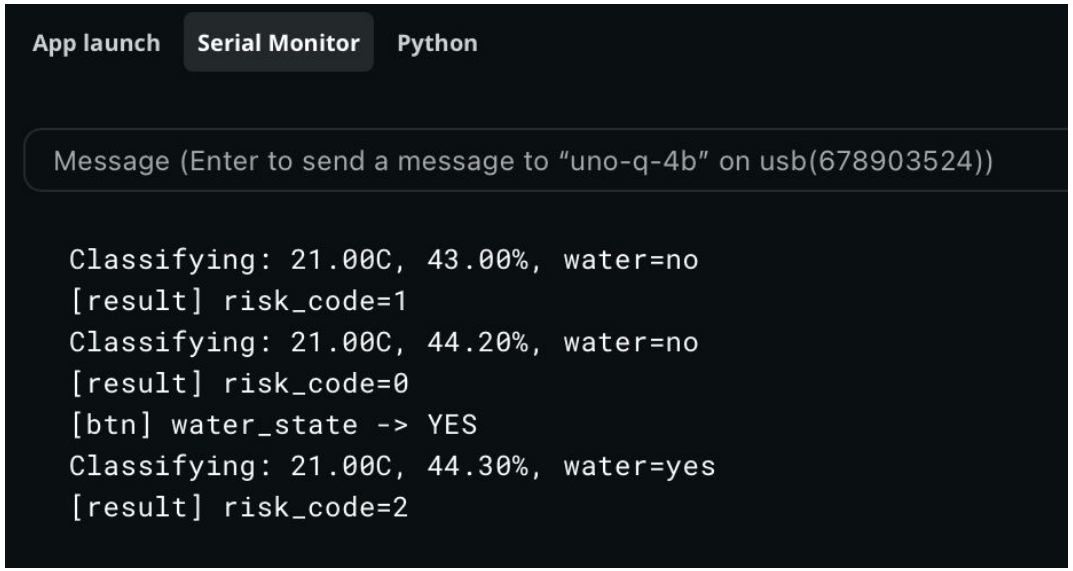
DEMO

Start it and watch

```
$ systemctl status llama-server --no-pager
$ curl -s http://127.0.0.1:8081/health

$ cd ~/ArduinoApps/risk-classifier
$ arduino-app-cli app start .      # first run: 2-3 min
$ arduino-app-cli app logs . --follow
```

> Check the model is up BEFORE starting the app. A dead endpoint looks like a Bridge bug.



The serial monitor

DEMO

How to drive it in class

1. **Power on**
   All LEDs blink three times, then green stays on.
2. **Press the button**
   The serial monitor prints water_state → YES. Press again to flip it back; the state holds between inferences.
3. **Wait for the cycle**
   Every 30 s the sketch fires an inference. All three LEDs come on while it thinks, then the verdict colour.
4. **Warm the sensor**
   Squeeze the DHT22 between your fingers and press the button. Ambient lab gives green; warm, humid, water gives red.

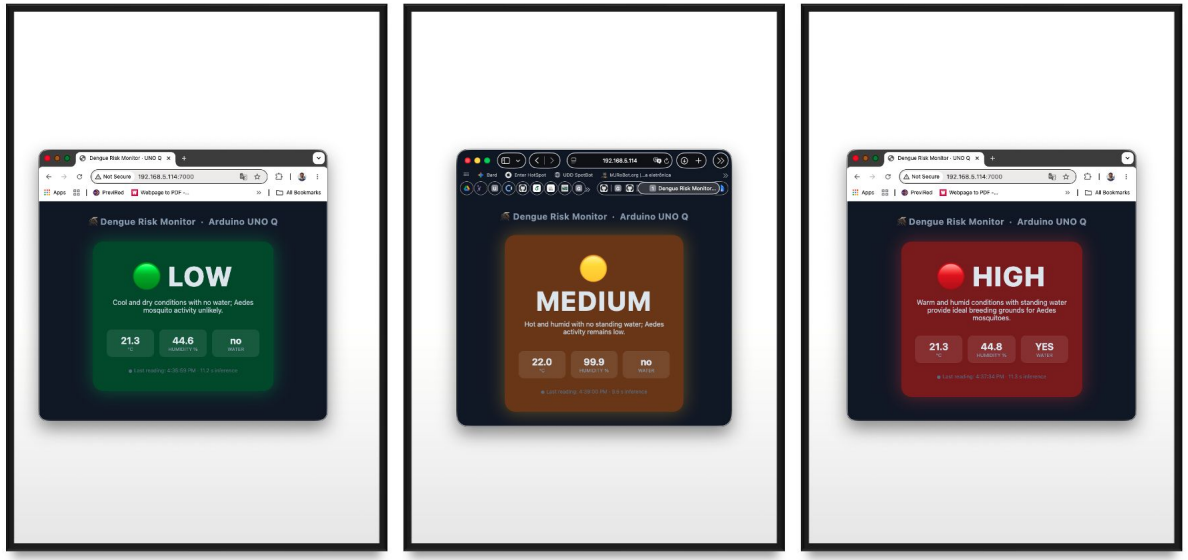
> Two real log lines: 24.3 °C, 38.7 % → low, 10.8 s · 27.6 °C, 87.8 % → high, 10.6 s

THE OFF-BOARD PATH

The same verdict, over HTTP

```
$ curl -X POST http://<UNO_Q_IP>:7000/classify \
    -H "Content-Type: application/json" \
    -d '{"temp_c": 30.1, "humidity_pct": 85,
         "standing_water": true}'

{ "risk": "high",
  "reason": "Warm humid conditions with standing
             water create ideal breeding habitat.",
  "latency_ms": 8234.7 }
```



A live dashboard is two more routes

Same classify() the MCU calls over Bridge. The Bridge gets a float; HTTP clients get the full JSON, including the reasoning and the measured latency.

STAGE 6 OF 6

Performance and choices

6

End-to-end this time — Bridge RPC, JSON parsing and prompt-building included.

MEASURED

End-to-end, on the full application

End-to-end classify()	6–12 s
Cold model load	~4 s
Idle RAM (server + app)	~700–800 MB
Model + context	~1,122 MiB
Generation throughput	~4.75 tok/s
CPU during decode	4 cores @ 100%
Temperature, inference	~54 °C
Power (max.)	3.1 W

6–12 s

per classify(), end to end

30 s

the polling window it sits inside

Ten seconds is invisible inside a thirty-second polling window — and completely unacceptable in a chat interface.

MODEL CHOICE

For this project, the 2B wins

Dimension	0.8B Q8_0	2B UD-Q4_K_XL
RAM footprint (model + 1024 ctx)	~1.1 GB	~1.9–2.0 GB
Free RAM after load (of ~3 GB)	~1.7 GB	~0.8 GB
Generation speed	~4.75 tok/s	~1.8–2.5 tok/s
End-to-end classify() latency	6–12 s	15–25 s (est.)
Quality on structured JSON	Good with a strong few-shot	Noticeably more robust

The opposite conclusion from Chapter 3 — and for a good reason: the extra 10 s disappears inside the polling window, and the JSON path gets more reliable.

WHEN IT BREAKS

The failures you will actually hit

1. **Bridge.call times out**
   A cold inference takes longer than the default Bridge timeout. Warm the model with one HTTP call after boot, or raise the timeout.
2. **Flask is not reachable**
   ports: [7000] missing from app.yaml — or the app was restarted rather than fully stopped and started after the edit.
3. **llama-server will not start**
   Check the model path in ExecStart, and journalctl for the real error. A typo in the GGUF filename is the usual cause.
4. **The verdict is right, the LED is wrong**
   A harness bug, not a model bug. Check RISK_CODE and the float rounding — verdict and actuator are two separate steps.

Also: OOM crashes (systemd restarts it, so the endpoint just works again 10 s later) and JSON parse failures (strengthen the few-shots first).

GOING FURTHER

Four directions from here

CHAPTER 5

Invert the flow

Register the MCU's capabilities as tools the model can call — 'read humidity', 'set LED colour'. Instead of the MCU calling Python, the model calls the MCU.

CHAPTER 3

Replace the button with a camera

The vision pathway runs on the same weights with a matching mmproj. The MCU triggers a capture, a vision-enabled classify() judges the frame, the LED logic is unchanged.

PRODUCTION

A trained classifier alongside

Edge Impulse when you want a trained model rather than zero-shot. A hybrid: the trained model handles the high-frequency case, the SLM handles the rare 'not sure'.

Also: edge RAG on the board — embed local documents and answer questions about them fully offline.

CARRY THESE OUT OF CHAPTER 4

Four things

1

A service, not a terminal

systemd is what makes the endpoint real: it starts at boot, restarts on failure, and nobody has to remember to launch it.

2

One logic path, two interfaces

Bridge gets a float the sketch can act on; HTTP gets the full JSON. Separate interface from logic and both stay simple.

3

Structured output is what makes it safe

response_format json_object plus two few-shots is what makes a 0.8B verdict reliable enough to drive a physical output.

4

Match the latency to the application

Ten seconds is invisible in a 30-second loop and unacceptable in a chat. Choose the shape the hardware is good at.

Small models, short outputs, batch-rate inference — and a physical output you can see from across the room.

RESOURCES

Where to read further

Resource	Where
`Arduino_RouterBridge library`	github.com/arduino-libraries/Arduino_RouterBridge
`Arduino App CLI`	github.com/arduino/arduino-app-cli
`llama.cpp HTTP server docs`	github.com/ggml-org/llama.cpp/blob/master/tools/server/README.md
`Unsloth Qwen3.5 inference guide`	unsloth.ai/docs/models/qwen3.5
`Flask documentation`	flask.palletsprojects.com
`Arduino UNO Q documentation`	docs.arduino.cc/hardware/uno-q

Next — Ch. 5, Agentic AI at the Edge: this chapter's classify() is one fixed call. Next, the model decides which tool to call, reads the result, and decides what to do next.

Questions?

Prof. Marcelo J. Rovai

rovai@unifei.edu.br

UNIFEI — Federal University of Itajubá, Brazil

AiEng4D — Academic Network Co-Chair

EdgeAIP — Academia-Industry Partnership Co-Chair